Sardar Vallabhbhai National Institute of Technology (SVNIT), Surat
Department of Artificial Intelligence
Agentic AI (AI465), Semester VII

Research Project — Phase 2

Implementation and Evaluation of the Approved Objective

Maximum marks: 20 | Individual work | Demonstration and presentation with a written report | Prerequisite: an objective approved in Phase 1.

1. Purpose

Phase 1 ended with one objective and an argument for why it is worth pursuing. Phase 2 asks you to carry it out: build the agent, run it against the baseline you named, and report honestly what happened. A result that fails to beat the baseline is a perfectly acceptable outcome, and in this field a well-measured negative result — that an elaborate agent does no better than a single well-prompted call at the same token budget — is often the more useful finding. What is not acceptable is an implementation that does not run, a comparison against no baseline, or a change of objective made silently.

2. Scope of the work

1. Implement exactly the objective approved for you at the end of Phase 1. The work is individual, as in Phase 1. A change of objective requires prior approval and a technical reason; reducing scope is allowed if you report what was dropped and why.
2. Build the baseline first and confirm that its numbers are in the same range as those published. If they are not, say so and investigate — a model version different from the paper's is the usual cause, and an unexplained gap is itself a finding worth reporting.
3. Then implement your contribution and run it under identical conditions: same model and version, same decoding settings, same task subset, same evaluation code.
4. Compare at a matched budget wherever your method costs more than the baseline. An agent that wins by spending nine times the tokens has not been shown to be better until the baseline has been given the same budget, for instance through self-consistency with more samples.
5. Run an ablation that isolates the single component you changed, so that the difference can be attributed to it rather than to incidental changes in the prompts.

3. Experimental protocol

These requirements apply to every project and are the difference between a demonstration and an experiment. They matter more here than in a conventional machine-learning project, because an LLM agent is stochastic, expensive per run and served by a model that may change under you.

Requirement	What is expected
Repetition	every reported number is the mean over at least three runs of the whole task subset, given with its standard deviation; a single run of a sampled agent is not a result
Pinned configuration	the exact model name and version string, the decoding temperature and top-p, the seed where the provider supports one, and the date of the runs, all recorded in the report and the repository
Prompts as artefacts	every prompt and system message is a file in the repository, not a string edited between runs; the baseline and your method differ in one thing only

Requirement	What is expected
Metrics	the task metric named in the objective, plus cost: model calls, prompt and completion tokens, and wall-clock latency per task
Traces	every run writes a full trace of prompts, tool calls, observations and outputs to disk, so that a failure can be opened and inspected rather than guessed at
Judging	if an LLM judges the output, the judge is named and its agreement with your own labels is checked on at least twenty tasks, and the judge should not be the model being graded
Reproducibility	the whole result table regenerates from a clean checkout with a documented command and a stated approximate cost

4. Deliverables

1. Source code with a README stating the environment, the install steps, the model and version used, the command that reproduces each table and figure, the expected runtime and the approximate token cost. API keys are never committed; the README says which environment variables are required.
2. The saved traces and the raw result files for the runs you report, so that a number in the table can be traced back to the run that produced it.
3. A report of at most four pages: restated objective; what was implemented; experimental setup including the pinned configuration; results table and plot; analysis including at least one failure case read from a trace; limitations and what you would do with four more weeks.
4. A slide deck of eight to ten slides and a live demonstration of the agent running on one task, or a recorded run of at most three minutes if the full experiment is too slow to show.
5. A presentation of eight minutes with four minutes of questions, including questions on the code.
6. Submit as RP2_Roll_No.zip containing the report, the slides, the code and the traces.

5. Use of external code and AI tools

You may use agent frameworks, published repositories and language models to help you write code. Three conditions apply. State in the README which parts are yours and which are borrowed, with a link for anything reused. Keep the borrowed part smaller than the part you wrote — wiring together a framework’s example scripts is not an implementation. Be able to explain any line of the submitted code when asked in the viva; code you cannot explain is treated as not submitted, and the marks for implementation are withheld regardless of whether the system runs.

6. Evaluation

Criterion	What earns full marks	Marks
Fidelity to the objective	the approved objective is what was built; deviations declared and justified	3
Working implementation	runs from a clean checkout on the documented command; baseline and proposed method both operational; failures handled rather than crashing	5
Experimental rigour	baseline reproduced, configuration pinned, at least three runs with variance, matched budget, ablation of the changed component, cost reported	5
Analysis	results interpreted rather than restated; at least one failure case traced to its cause; limitations stated without prompting	4
Demonstration and viva	demo works, report is clear, questions on the code and on the results answered correctly	3

7. Schedule

Week	Milestone
5	environment set up, model access working, benchmark subset fixed and stored
6	baseline running end to end and its numbers reported in the lab slot
7	contribution implemented; first comparative run with traces saved
8	ablation and repeated runs complete; cost and latency measured; report drafted
9	Phase 2 demonstration, presentation and submission

8. Where marks are commonly lost

1. A single run per configuration, so a difference of two points cannot be told apart from sampling noise.
2. No cost reported, so a method that spends ten times the tokens is presented as a clean win.
3. The baseline given one call and the proposed agent given many, with no matched-budget comparison.
4. An LLM judge from the same model as the system under test, never checked against any human label.
5. Prompts edited by hand between runs, so the two conditions differ in more than the one component claimed.
6. The model version left unstated, making the result impossible to repeat once the endpoint is updated.
7. A demonstration that shows a chat interface but never the experiment the objective promised.